

Guide de création d'une skill Claude Code

De l'idée au déploiement : workflow complet pour écrire des skills qui se déclenchent au bon moment, restent maintenables et consomment le minimum de tokens.

Avant de commencer : est-ce vraiment une skill ?

Répondez à ces 3 questions :

1. La tâche est-elle répétitive ? (au moins 3 fois par semaine pour l'équipe)
2. La tâche a-t-elle une structure définie ? (pas juste "réponds bien")
3. La tâche dépasse-t-elle ce que Claude sait faire sans instructions ?

Si vous répondez OUI aux 3 → créez une skill. Si NON à l'une → ajoutez plutôt une règle dans `CLAUDE.md` ou répondez directement.

Étape 1 — Définir le périmètre

Remplir la fiche de cadrage (avant d'écrire une seule ligne)

Nom candidat : _____
But en une phrase : _____
Déclencheur principal (ce que dit l'utilisateur) : _____
Déclencheurs secondaires : _____
Ce que la skill NE fait PAS : _____
Outils nécessaires : _____
Fichiers de référence nécessaires : _____

Exemple rempli :

Nom candidat : reviewing-code-changes
But : Relire du code modifié et produire un rapport priorisé
Déclencheur principal : "fais une revue de code", "code review", "check ce diff"
Déclencheurs secondaires : "est-ce correct ?", "vérifie avant merge", "regarde ce que j'ai écrit"
Ce que la skill NE fait PAS : ne modifie pas le code, ne résout pas les bugs
Outils : Read, Grep, Glob (lecture seule)
Fichiers de référence : checklists-par-langage.md

Étape 2 — Écrire la description (le champ le plus important)

La description est la **seule chose que Claude lit** pour décider de déclencher la skill. Investissez du temps ici.

Recette d’une bonne description

[Ce que la skill fait] + [contextes de déclenchement] + [contre-exemples si besoin]

Gabarit :

description: >-
[Verbe à la 3ème personne] [objet] pour [bénéfice].
À utiliser quand [contexte 1], [contexte 2], ou [contexte 3].
Même si l'utilisateur ne dit pas explicitement [terme].

Exemple :

description: >-
Relit du code modifié (diff Git, PR, ou fichiers ciblés) pour repérer bugs, failles de sécurité, cas limites et écarts aux conventions.
À utiliser dès que l'utilisateur demande une revue de code, un code review, de vérifier un diff ou une PR avant merge, ou de relire une fonction – même s'il ne dit pas explicitement le mot "revue".

Tests de la description

Avant de valider, posez-vous ces questions :

Test	Réponse souhaitée
Un humain qui lit la description comprend-il ce que fait la skill ?	Oui
Les mots-clés que l'utilisateur utiliserait y sont-ils ?	Oui
La description dit-elle QUAND utiliser (pas seulement QUOI) ?	Oui
Fait moins de 1024 caractères ?	Oui
Rédigée à la 3ème personne ?	Oui

Étape 3 — Choisir les outils (allowed-tools)

Principe du moindre privilège : accordez le strict minimum.

Type de skill	Outils typiques
Analyse / revue (lecture seule)	Read, Grep, Glob
Génération de code	Read, Grep, Glob, Write
Automatisation de tâches	Read, Write, Bash
Recherche web	WebSearch, WebFetch, Read
Interaction GitHub	mcp__github__* (spécifier les actions)

Dans le frontmatter – n'incluez que ce dont vous avez réellement besoin
`allowed-tools: Read, Grep, Glob`

Étape 4 — Structurer le corps du `SKILL.md`

Ordre recommandé des sections

1. Titre + but (1 ligne)
2. Quand l'utiliser / Quand NE PAS l'utiliser (clarté du périmètre)
3. Workflow avec checklist (le moteur de la skill)
4. Règles & anti-patterns (ce qui fait la différence qualitative)
5. Format de sortie (gabarit strict si applicable)
6. Exemples (2–3 paires entrée → sortie)
7. Ressources liées (renvoi vers `references/`)

Le workflow : la section la plus importante

Un bon workflow dit à Claude **quoi faire dans quel ordre**, pas pourquoi. Format checklist que Claude recopie et coche :

```
[Nom de la skill] :  
- [ ] 1. Glob – identifier les fichiers concernés  
- [ ] 2. Grep – localiser les définitions et usages  
- [ ] 3. Read – lire les sections pertinentes uniquement  
- [ ] 4. Analyser selon les axes : correction, sécurité, lisibilité  
- [ ] 5. Rédiger le rapport au format imposé
```

Pourquoi la checklist ? Claude la recopie au début de sa réponse. Cela crée un ancrage visible qui réduit les oublis et permet à l'utilisateur de suivre la progression.

Étape 5 — Divulcation progressive : quoi mettre où

<code>SKILL.md</code> (niveau 2)	– instructions d'action, < 500 lignes
<code>references/fichier.md</code> (niv 3)	– détails, checklists longues, specs
<code>scripts/script.py</code> (niv 3)	– code exécutable, 0 token si non lu
<code>assets/</code> (niveau 3)	– gabarits, exemples lourds

Règle de décision :

“Est-ce que Claude a besoin de ça à chaque déclenchement de la skill ?”

- Oui → dans `SKILL.md`
- Non → dans `references/`

Exemple : une checklist de 50 points par langage → `references/checklists-par-langage.md` . Dans `SKILL.md` : juste "Pour les règles précises, lis `references/checklists-par-langage.md`" .

Étape 6 — Le nom de la skill

Règles du nom

Règle	Exemple correct	Exemple incorrect
Minuscules uniquement	<code>reviewing-code</code>	<code>ReviewingCode</code>
Tirets uniquement	<code>processing-pdfs</code>	<code>processing_pdfs</code>
Forme gérondif	<code>analyzing-logs</code>	<code>analyze-logs</code> , <code>log-analyzer</code>
Max 64 caractères	<code>generating-release-notes</code>	✓
Pas de <code>claude</code> ni <code>anthropic</code>	<code>reviewing-code</code>	<code>claude-reviewer</code>
Pas de termes vagues	<code>reviewing-code-changes</code>	<code>helper</code> , <code>utils</code> , <code>tools</code>

Le nom devient la slash command

`name: reviewing-code-changes` → `/reviewing-code-changes` dans Claude Code.

Étape 7 — Tester la skill

Test minimal (3 prompts)

Testez avec des formulations variées de la même demande :

Prompt 1 (formulation directe) :
"Fais une revue de code de ce fichier"

Prompt 2 (formulation indirecte) :
"Est-ce que ce code est correct avant que je merge ?"

Prompt 3 (formulation sans le mot-clé) :
"Regarde ce que j'ai écrit, je veux être sûr"

Attendu : la skill se déclenche pour les 3.

Test de non-déclenchement

Prompt à NE PAS déclencher :
"Comment fonctionne `async/await` ?"
"Quel est le meilleur ORM pour Python ?"

Attendu : Claude répond directement sans charger la skill.

Grille d'évaluation

- [] La skill se déclenche sur les formulations directes
- [] La skill se déclenche sur les formulations indirectes
- [] La skill NE se déclenche PAS sur les questions non pertinentes
- [] Le workflow est suivi dans l'ordre
- [] Le format de sortie est respecté
- [] Les fichiers references/ sont chargés au bon moment
- [] La réponse tient dans le contexte disponible
- [] Aucun outil non listé dans allowed-tools n'est utilisé

Étape 8 — Emplacement et déploiement

Skill locale (projet)

```
.claude/  
└─ commands/  
    └─ reviewing-code-changes/  
        └─ SKILL.md  
        └─ references/  
            └─ checklists-par-langage.md
```

→ Disponible uniquement dans ce projet. Se commite avec le repo.

Skill globale (utilisateur)

```
~/.claude/  
└─ commands/  
    └─ reviewing-code-changes/  
        └─ SKILL.md
```

→ Disponible dans tous vos projets. Ne se commite pas.

Skill d'équipe (via package)

Voir `skills/banques-de-skills.md` pour la distribution via npm, Git submodules, ou symlinks.

Checklist finale avant livraison

Frontmatter

- [] name : minuscules-tirets, gérondif, max 64 car., sans claude/anthropic
- [] description : < 1024 car., 3^e personne, QUOI + QUAND, mots-clés présents
- [] allowed-tools : limité au strict nécessaire

Corps

- [] < 500 lignes au total
- [] Workflow présent avec checklist numérotée
- [] Au moins 2 règles  /  concrètes
- [] Format de sortie imposé (si applicable)
- [] 1-2 exemples entrée/sortie concrets
- [] Références vers references/ pour les détails (pas dans le corps)

Qualité

- [] Pas d'explications que Claude sait déjà
- [] Terminologie cohérente (un mot par concept)
- [] Chemins en / (jamais \)
- [] Testé sur 3 prompts réalistes

Ressources liées

- Template vierge à copier : `templates/skill/SKILL.md`
- Exemple concret (revue de code) : fourni dans les uploads de démarrage
- Cycle de vie et mises à jour : `skills/cycle-de-vie.md`
- Organisation en banques : `skills/banques-de-skills.md`